

# გაკვეთილი 11 mod 3 (3.2)

## ივენტები (Events) — სრული თეორია

---

### 1. რა არის JavaScript-ის როლი ვებგვერდში

ვებგვერდი ძირითადად 3 ნაწილისგან შედგება:

#### HTML

ქმნის გვერდის სტრუქტურას  
მაგალითად:

- სათაური
- ტექსტი
- ლილაკი
- სურათი

#### CSS

აკეთებს დიზაინს  
მაგალითად:

- ფერი
- ზომა
- ჩარჩო
- განლაგება

#### JavaScript

ამატებს მოქმედებას და ინტერაქტიულობას  
მაგალითად:

- ლილაკზე დაჭერისას რაღაც მოხდეს
  - ტექსტი შეიცვალოს
  - ფერი შეიცვალოს
  - ელემენტი დაიმალოს
  - მაუსზე რეაგირება მოხდეს
-

## 2. რა არის DOM

როცა ბრაუზერი HTML-ს კითხულობს, ის გვერდის ყველა ელემენტს ქმნის როგორც ობიექტებს.

ამ სისტემას ეწოდება **DOM**.

DOM ნიშნავს:

### Document Object Model

მარტივად რომ ვთქვათ:

- 👉 HTML ელემენტები JavaScript-სთვის ხდება ხელმისაწვდომი
- 👉 ანუ JS-ს შეუძლია გვერდის ელემენტები იპოვოს, შეცვალოს და მართოს

მაგალითად, თუ HTML-ში გაქვს:

Hello

JavaScript-ს შეუძლია ამ ელემენტთან მუშაობა.

---

## 3. რა არის document

`document` არის ობიექტი, რომელიც წარმოადგენს მთლიან HTML გვერდს.

ანუ როცა ვწერთ:

`document`

ვგულისხმობთ მთელ ვებგვერდს.

ამ `document` -ის საშუალებით შეგვიძლია:

- ელემენტის პოვნა
  - ტექსტის შეცვლა
  - სტილის შეცვლა
  - event-ის დამატება
  - ახალი ელემენტის შექმნა
- 

## 4. რა არის `getElementById()`

ეს არის ერთ-ერთი ყველაზე მნიშვნელოვანი მეთოდი.

## მნიშვნელობა

`getElementById()` ეძებს HTML ელემენტს მისი `id`-ით.

თუ HTML-ში გაქვს:

Hello

JavaScript-ში შეგვიძლია ასე წამოვიღოთ:

```
let text = document.getElementById("demo");
```

აქ:

- `document` → მთელი გვერდი
- `getElementById("demo")` → მოძებნე ელემენტი, რომელსაც `id` არის `demo`

---

## 5. რატომ გვჭირდება `getElementById()`

იმიტომ რომ სანამ JavaScript რამეს შეცვლის, ჯერ უნდა იპოვოს კონკრეტული ელემენტი.

მაგალითად თუ გვინდა:

- ტექსტის შეცვლა
- ფერის შეცვლა
- ღილაკზე `event`-ის დამატება
- `div`-ის დამალვა

ჯერ ელემენტი უნდა წამოვიღოთ.

---

## 6. `id` რა არის

`id` არის HTML ელემენტის უნიკალური სახელი.

მაგალითი:

Click

აქ ამ ღილაკის `id` არის `"btn"`.

მერე JavaScript-ით ვწერთ:

```
let button = document.getElementById("btn");
```

და ახლა `button` ცვლადში გვაქვს ეს კონკრეტული ღილაკი.

---

## 7. მაგალითი — `getElementById()`

Hello

### რა მოხდება

JavaScript იპოვის იმ `<p>` ელემენტს, რომელსაც აქვს `id="text"`.

### Output კონსოლში

კონსოლში გამოჩნდება HTML ელემენტი, დაახლოებით ასე:

Hello

---

## 8. მაგალითი — ტექსტის შეცვლა

Hello

### რა ხდება

- JS პოულობს `<p>` ელემენტს
- მერე `textContent` -ით ცვლის მის ტექსტს

### თავიდან გვერდზე წერია

Hello

### შემდეგ ხდება

Hi

---

## 9. რა არის Event

ახლა მთავარ თემაზე გადავიდეთ.

**Event** ნიშნავს მოვლენას, ქმედებას, მოქმედებას, რაც ხდება გვერდზე.

მაგალითად:

- მომხმარებელმა დააჭირა ღილაკს
- მაუსი მიიტანა ტექსტზე
- კლავიატურაზე ღილაკი დააჭირა
- გვერდი ჩაიტვირთა
- input-ში რაღაც ჩაწერა

ეს ყველაფერი არის **event**.

---

## 10. event-ის ძალიან მარტივი ახსნა

შეგვიძლია ასე წარმოვიდგინოთ:

**რაღაც მოხდა** → პროგრამამ რეაგირება მოახდინა

მაგალითად:

- მოხდა click → ტექსტი შეიცვალა
- მოხდა mouseover → ფერი შეიცვალა
- მოხდა keydown → კონსოლში ტექსტი დაიბეჭდა

ანუ event არის **სიგნალი**, რომ რაღაც მოხდა.

---

## 11. რატომ არის event მნიშვნელოვანი

event-ების გარეშე გვერდი იქნება უბრალოდ სტატიკური.

ანუ მხოლოდ აჩვენებს ტექსტს და სურათს.

event-ების საშუალებით გვერდი ხდება **ცოცხალი და ინტერაქტიული**.

მაგალითად:

- ღილაკზე დაჭერისას რაღაც იცვლება
- მენიუ იხსნება
- ტექსტი იცვლება
- ფერი იცვლება

- ელემენტი იმალება
- 

## 12. Event როგორ მუშაობს

ძირითადი იდეაა:

ელემენტი + მოვლენა + მოქმედება

მაგალითად:

- ელემენტი = ღილაკი
  - მოვლენა = click
  - მოქმედება = ტექსტის შეცვლა
- 

## 13. Event-ის ძირითადი ნაწილები

როცა event-ს ვამატებთ, გვჭირდება:

### 1. ელემენტი

რომელ ელემენტზე უნდა იმუშაოს event-მა

### 2. event-ის ტიპი

მაგალითად:

- click
- mouseover
- mouseout
- keydown

### 3. ფუნქცია

რა უნდა გაკეთდეს როცა event მოხდება

---

## 14. Event-ის დამატების გზები

event-ის დამატების რამდენიმე გზა არსებობს, მაგრამ სწავლებისთვის და პრაქტიკულად ყველაზე მნიშვნელოვანია 2:

## 1) HTML ატრიბუტით

მაგალითად:

Click

## 2) JavaScript-ში `addEventListener()`

ეს არის ყველაზე სწორი და თანამედროვე გზა.

---

## 15. რატომ ჯობს `addEventListener()`

იმიტომ რომ:

- უფრო სუფთა კოდია
  - HTML და JS ცალკეა
  - უკეთესი სტილია
  - ერთ ელემენტზე რამდენიმე event-ის დამატება შეგვიძლია
  - რეალურ პროექტებში ამას უფრო ხშირად იყენებენ
- 

## 16. `addEventListener()` სინტაქსი

```
element.addEventListener("eventName", function() {  
  // კოდი  
});
```

მაგალითად:

```
button.addEventListener("click", function() {  
  console.log("Clicked");  
});
```

აქ:

- `button` → ელემენტი
- `"click"` → მოვლენა
- `function() { ... }` → რა უნდა მოხდეს

---

## 17. პირველი სრული მაგალითი — click event

დააჭირე

### რა ხდება აქ

1. HTML-ში გვაქვს ღილაკი
2. `getElementById("btn")` -ით ვპოულობთ ამ ღილაკს
3. `addEventListener("click", ...)` -ით ვამბობთ:  
როცა ამ ღილაკზე click მოხდება, გაეშვას ფუნქცია
4. ფუნქციაში ვბეჭდავთ ტექსტს

### Output

ღილაკზე დაჭერისას კონსოლში დაიბეჭდება:

ღილაკზე მოხდა დაჭერა

---

## 18. event-ზე ტექსტის შეცვლა

ძველი ტექსტი

შეცვლა

### რა ხდება

- თავიდან პარაგრაფში წერია: ძველი ტექსტი
- როცა ღილაკზე დააჭერ
- პარაგრაფის ტექსტი შეიცვლება

### თავიდან გვერდზე წერია

ძველი ტექსტი

### click-ის შემდეგ

ახალი ტექსტი

---



## 19. event-ზე ფერის შეცვლა

Hello

ფერის შეცვლა

### რა ხდება

ლილაკზე დაჭერისას ტექსტის ფერი გახდება წითელი.

### თავიდან

ტექსტი ჩვეულებრივი ფერისაა.

### click-ის შემდეგ

ტექსტი იქნება:

Hello

მაგრამ ფერი იქნება წითელი.

---

## 20. event-ზე ელემენტის დამალვა

ეს არის ტექსტი

დამალვა

### რა ხდება

ლილაკზე დაჭერისას div გაქრება.

### თავიდან

გვერდზე ჩანს:

ეს არის ტექსტი

### click-ის შემდეგ

ელემენტი აღარ გამოჩნდება.

---

## 21. mouseover event

`mouseover` მუშაობს მაშინ, როცა მაუსი გადავა ელემენტზე.

მოიტანე მაუსი აქ

## რა ხდება

როცა მაუსი გადავა ამ ტექსტზე, ტექსტი გახდება ლურჯი.

## თავიდან

ტექსტი ჩვეულებრივი ფერისაა.

## მაუსის მიტანისას

ტექსტი გახდება ლურჯი.

---

## 22. mouseout event

`mouseout` მუშაობს მაშინ, როცა მაუსი გადის ელემენტიდან.

მოიტანე მაუსი აქ

## რა ხდება

- მაუსის მიტანისას → ტექსტი ლურჯდება
  - მაუსის გაწევისას → ისევ შავდება
- 

## 23. mouseover + mouseout პალიან მნიშვნელოვანი ახსნა

ეს ორი ხშირად ერთად გამოიყენება.

მაგალითად:

- ღილაკზე მაუსის მიტანისას დიზაინი შეიცვალოს
- ელემენტი გააქტიურდეს
- `hover` ეფექტი გაკეთდეს JavaScript-ით

თუმცა ხშირად ასეთ `hover` ეფექტებს CSS-ით აკეთებენ, მაგრამ სასწავლოდ JS-ითაც კარგია.

---

## 24. dblclick event

`dblclick` ნიშნავს ორმაგ დაჭერას.

ორმაგად დააჭირე

### Output გვერდზე

თავიდან:

ორმაგად დააჭირე

ორმაგი დაჭერის შემდეგ:

ორმაგი click მოხდა

---

## 25. keydown event

`keydown` ხდება მაშინ, როცა კლავიატურაზე ღილაკი დაიჭირება.

### რა ხდება

კლავიატურაზე ნებისმიერ ღილაკზე დაჭერისას კონსოლში დაიბეჭდება:

ღილაკი დაიჭირა

---

## 26. keyup event

`keyup` ხდება მაშინ, როცა კლავიატურის ღილაკს გავუშვებთ.

### Output

ღილაკის გაშვებისას:

ღილაკი გაუშვა

---

## 27. keydown და keyup განსხვავება

### keydown

მუშაობს დაჭერის მომენტში

## keyup

მუშაობს გაშვების მომენტში

---

## 28. load event

`load` ხდება მაშინ, როცა გვერდი სრულად ჩაიტვირთება.

### Output

გვერდის ჩატვირთვისას:

გვერდი ჩაიტვირთა

---

## 29. event-ზე ფუნქციის ცალკე დაწერა

event-ის დროს ფუნქცია შეიძლება პირდაპირ შიგნით დავწეროთ, ან ცალკე.

### inline ფუნქცია

```
button.addEventListener("click", function() {  
  console.log("Hello");  
});
```

### ცალკე ფუნქცია

Click

### Output

დილაკზე დაჭერისას:

Hello

---

## 30. რატომ არის ცალკე ფუნქცია კარგი

- კოდი უფრო სუფთაა

- მრავალჯერ გამოყენება შეიძლება
  - ახსნა მოსწავლეებისთვისაც უფრო ადვილია
  - დიდი პროექტებში უკეთესია
- 

## 31. event object — მოკლე თეორია

როცა event ხდება, JavaScript ხშირად გვაწვდის დამატებით ინფორმაციას ამ მოვლენაზე. ეს არის **event object**.

მაგალითად:

```
button.addEventListener("click", function(event) {  
  console.log(event);  
});
```

აქ event შეიცავს ინფორმაციას:

- რომელ ელემენტზე მოხდა მოვლენა
- რა ტიპის მოვლენა იყო
- სად მოხდა
- რა ღილაკი დააჭირეს და ა.შ.

ეს ნაწილი შეგიძლია ძალიან ღრმად ჯერ არ შეხვიდე, მაგრამ სახელად მაინც უნდა იცოდნენ.

---

## 32. ყველაზე მნიშვნელოვანი event-ები ამ ეტაპზე

ამ დონეზე ყველაზე კარგია ახსნა:

- click
- dblclick
- mouseover
- mouseout
- keydown
- keyup
- load

ესენი სრულიად საკმარისია მოსწავლეებისთვის საწყის ეტაპზე.

---

## 33. ერთი დიდი მაგალითი

Hello

ტექსტის შეცვლა

ფერის შეცვლა

დამალვა

თავიდან გვერდზე წერია

Hello

პირველ ღილაკზე დაჭერისას

გამოჩნდება:

New text

მეორე ღილაკზე დაჭერისას

ტექსტი გახდება მწვანე.

მესამე ღილაკზე დაჭერისას

ტექსტი საერთოდ გაქრება.

---

## 34. მოსწავლემ რა უნდა გაიგოს თეორიულად

აუცილებლად უნდა ესმოდეს:

### 1. Event რა არის

მოვლენა, რაც ხდება გვერდზე

### 2. ვინ იწვევს event-ს

ხშირად მომხმარებელი:

- click
- მაუსი
- კლავიატურა

### 3. პროგრამა რას აკეთებს

რეაგირებს event-ზე

### 4. JavaScript როგორ პოულობს ელემენტს

```
document.getElementById()
```

### 5. event როგორ ემატება

```
addEventListener()
```

### 6. event-ის დროს რა შეგვიძლია

- ტექსტის შეცვლა
- ფერის შეცვლა
- ელემენტის დამალვა
- კონსოლში შეტყობინების გამოტანა

---

## 35. ტიპური შეცდომები

### შეცდომა 1 — არასწორი id

შეცდომა 2 — script ელემენტზე ადრე იწერება

თუ JS იმუშავებს მანამდე, სანამ ელემენტი ჩაიტვირთება, შეიძლება ვერ იპოვოს.

ამიტომ ხშირად script ბოლოში იწერება.

შეცდომა 3 — ფრჩხილების არევა

მაგალითად addEventListener -ში ბრჭყალები, ფრჩხილები და ფიგურული ფრჩხილები სწორად უნდა იყოს.

---

## 36. სავარჯიშოები

### ძალიან მარტივი

1

შექმენი ღილაკი. დაჭერისას კონსოლში დაიბეჭდოს:

Clicked

**2**

შექმენი პარაგრაფი და ღილაკი. ღილაკზე დაჭერისას პარაგრაფის ტექსტი შეიცვალოს.

**3**

შექმენი ტექსტი და ღილაკი. ღილაკზე დაჭერისას ტექსტის ფერი გახდეს წითელი.

**4**

შექმენი `div` და ღილაკი. ღილაკზე დაჭერისას `div` გაქრეს.

**5**

შექმენი პარაგრაფი. მაუსის მიტანისას მისი ფერი გახდეს ლურჯი.

---

## მარტივი

**6**

მაუსის გაწევისას ფერი დაბრუნდეს შავზე.

**7**

ორმაგ დაჭერაზე ტექსტი შეიცვალოს.

**8**

კლავიატურაზე ღილაკის დაჭერისას კონსოლში დაიბეჭდოს:

`Key pressed`

**9**

ღილაკზე დაჭერისას ტექსტის ზომა გაიზარდოს.

**10**

ღილაკზე დაჭერისას `background color` შეიცვალოს.

---

## საშუალო



## 11

ორი ღილაკი შექმენი:

- ერთი ცვლის ტექსტს
- მეორე ცვლის ფერს

## 12

სამი ღილაკი შექმენი:

- hide
- show
- color change

## 13

გვერდის ჩატვირთვისას კონსოლში დაიბეჭდოს:

Page loaded

## 14

ერთ ელემენტზე დაამატე:

- mouseover
- mouseout

## 15

კლავიატურის დაჭერაზე ტექსტი შეიცვალოს:

You pressed a key

---

## ცოტა რთული

## 16

ღილაკზე ყოველი დაჭერისას გაიზარდოს რიცხვი.

მაგალითად თავიდან იყოს:

0

პირველ click-ზე:

1

მეორეზე:

2

**17**

შექმენი ღილაკი, რომელიც ტექსტს ხან მალავს და ხან აჩენს.

**18**

შექმენი ელემენტი, რომელსაც click-ზე ფერი უცვლება.

**19**

ორმაგი click-ზე ელემენტი გაქრეს.

**20**

შექმენი პატარა interactive box, სადაც:

- mouseover-ზე ფერი იცვლება
  - mouseout-ზე უბრუნდება ძველ ფერს
  - click-ზე ტექსტი იცვლება
- 

## **37. საკლასო მინი-პროექტები**

### **პროექტი 1 — Color changer**

ერთი ტექსტი + 3 ღილაკი:

- წითელი
- მწვანე
- ლურჯი

### **პროექტი 2 — Text changer**

ერთი პარაგრაფი + 2 ღილაკი:

- ტექსტის შეცვლა
- ტექსტის წაშლა

## პროექტი 3 — Hide / Show

`div` + ორი ღილაკი:

- Hide
- Show

## პროექტი 4 — Counter

ღილაკზე ყოველი დაჭერისას რიცხვი იზრდება

## პროექტი 5 — Mouse effect

`div` , რომელზეც:

- მაუსის მიტანისას ფერი შეიცვალოს
- გაწევისას დაბრუნდეს

---

## 38. ძალიან მოკლე შეჯამება

### `document`

წარმოადგენს მთელ HTML გვერდს

### `getElementById()`

პოულობს ელემენტს `id`-ით

## Event

არის მოვლენა, რაც ხდება გვერდზე

### `addEventListener()`

`event`-ს ამატებს ელემენტზე

## ძირითადი event-ები

- click
- dblclick
- mouseover
- mouseout
- keydown

- keyup
- load